The Importance of Memory Management

Memory management requires that the programmer provides ways to dynamically allocate portions of memory to programs, when requested, and free it for reuse when it is no longer needed. In any advanced computer system, where more than a single process might be running at any given point in time, this is critical.

When any program is loaded into the memory, it is organised into three parts called segments, which are:

- Text segment (the code segment)
- Stack segment
- Heap segment

Text segment

The text segment is where the compiled code of the program itself resides. This is the machine language representation of the program steps to be carried out, including all functions making up the program, both user and system defined. So, in general, an executable program generated by a compiler (like GCC) will have memory organisation, as shown in Figure 1. This segment is further categorised as follows.

Code segment: This contains the code (executable or code binary).

Data segment: This is sub-divided into two parts.

- *Initialised data segment:* All the global, static and constant data are stored in the data segment.
- *Uninitialised data segment:* All the uninitialised data are stored in the Block Started by Symbol (BSS).

Stack segment

The stack is used to store our local variables, and for passing arguments to the functions along with the return address of the instruction that is to be executed after the function call is over. When a new stack frame needs to be added (as a result of a newly called function), the stack grows downwards.